TECH SHARING · ON-DEVICE INFERENCE × AGENTS

K3 时代

开源大模型端侧推理性能优化与 Agents 协作

从单机加速到多智能体编排的工程实践

2026.08 · 60 min 技术分享 · 48 页

覆盖模型：Kimi K3 / K2 · Qwen3 · DeepSeek-V3 / R1 · Llama 4

覆盖技术：TurboQuant 量化 · KV Cache 分层 · PD 分离 · 大小脑协作 · MCP / A2A

端侧推理 × Agents 协作

目录

CONTENTS

共 48 页
引言 + 5 章 + 结语
60 min · 10/20/15/10/5

引言　从 Vibe Coding 到 Agent 协作
P03–07　什么是 Vibe Coding · 四阶段演化 · 开发者新角色 · 要不要读代码

CH1　K3 时代：开源大模型的范式跃迁
P08–12 · 10 min　背景 · K3 架构革新 · 不可能三角 · 四重约束

CH2　端侧推理性能优化的核心技术栈
P13–25 · 20 min　技术栈全景 · 量化 · KV Cache · 推测解码 · 引擎 · 实测

CH3　端侧 Agents 协作架构设计
P26–41 · 15 min　为什么协作 · 大小脑 · 安全与双因子 · 协作模式 · ReAct & Handoff · 端云拓扑

CH4　工程实践与落地案例
P42–45 · 10 min　落地案例 · 性能数据与踩坑 · 路线图

CH5　未来展望与开源计划
P46 · 5 min　K3 演进 · 端侧 Agent 生态 · 开源计划

结语　47 三条判断收束　48 参考来源　P47–48

CH0 · INTRODUCTION

00

引言：从 Vibe Coding 到 Agent 协作

2025 年 vibe coding 兴起：自然语言成为新的编程入口，「写代码」这件事开始被整体外包给模型；

三年之间，开发者从「不写代码」走向「不看代码」——本章交代这条演化线，以及它成立的两个工程前提。

P04 什么是 Vibe Coding · P05 四阶段演化 · P06 开发者新角色 · P07 要不要读代码

Vibe Coding：完全交给氛围，忘记代码的存在

词的出处：Andrej Karpathy 原帖（2025-02-02）

> *"There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists."*
>
> *"I 'Accept All' always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it."*

中文释义：彻底顺着感觉走、拥抱指数级增长的能力，**连「代码存在」这件事都忘掉**——出错就把报错原样贴回去，通常就好了。

—— @karpathy，X（2025-02-02），原帖浏览约 450 万

年度词汇

Collins 柯林斯词典将 "vibe coding" 评为 **2025 年度词汇**（2025-11-06）：「用自然语言提示 AI 辅助编写代码」。

25% × 95%+

YC 管理合伙人 Jared Friedman（2025-03）：W25 批次 **25% 初创公司的代码库 95% 以上由 AI 生成**，且这些创始人技术能力本足以手写全部代码。

来源：x.com/karpathy（2025-02-02）· aa.com.tr（Collins 年度词汇，2025-11-06）· TechCrunch（YC W25，2025-03-06）

从不写代码，到不看代码：AI 编程的四阶段演化

关键跃迁：交接的是「看代码」这件事

① AI 补全

GitHub Copilot 技术预览 · 2021-06

人写人看，AI 只补片段——IDE 里的「超级 Tab」。

人的角色：作者，每一行都经手

② 对话式编程

ChatGPT 2022-11 · Cursor 2023-03

人描述需求、AI 起草，但修改与整合仍由人动手。

人的角色：作者，边问边改

③ Vibe Coding

Karpathy 命名 · 2025-02

不写代码、Accept All，但出错仍要人 review 兜底。

人的角色：reviewer

④ Agentic Coding

Devin 2024-03 · Claude Code 2025-02 · Codex 云端 agent 2025-05

不看代码：任务整包交给 agent，人只做验收与指挥。

人的角色：orchestrator

能力佐证：SWE-bench 解决率四年演进

≈1.7%	13.86%	49%	70.3%
GPT-4 裸模型基线（2023）	Devin（2024-03，随机 25% 子集）	Claude 3.5 Sonnet（2024-10，Verified）	Claude 3.7 Sonnet（2025，Verified）

评测口径：SWE-bench Verified（人工筛选子集）；Claude 系列含 agentic scaffold，GPT-4 为无 scaffold 基线，Devin 为无辅助随机子集——跨口径数字仅用于趋势判断。

来源：github.blog（Copilot，2021-06） · openai.com（ChatGPT，2022-11） · cognition.ai（Devin，2024-03） · anthropic.com（Claude Code，2025-02） · openai.com（Codex，2025-05） · skywork.ai · arXiv:2506.03921

敢不看代码的前提：协作机制可信，推理成本够低

开发者角色迁移链

- **author**：亲手写每一行，理解每一行
- **reviewer**：AI 起草，人审查、纠偏、兜底
- **orchestrator**：不看实现，只做验收与指挥

> *"The hottest new programming language is English."*
>
> —— @karpathy，X（2023-01-24），比 vibe coding 早两年的预告

> *「生成一个错误实现的成本几乎为零，但审查和纠正它的成本不是。」*
>
> —— IBM iX，转引自 paraphraselabs.com（2026-07）

全篇地图：两个前提，两条主线

「跑得动」：端侧推理性能优化 → CH1–CH2（P08–25）

推理成本够低：量化、KV Cache、推测解码、引擎与硬件适配，让 K3 级模型在端侧可用。

「敢托付」：Agents 协作架构与工程实践 → CH3–CH4（P26–45）

协作机制可信：编排模式、验证闭环、协议化接口与端云分工，让「不看代码」可控。

展望 → CH5（P46）

端侧推理与 Agents 协作汇合之后的工程版图。

本讲的全部内容，就是把这两个「敢」的前提逐项落成工程方案。

来源：twitter.com/karpathy（2023-01-24） · paraphraselabs.com（IBM iX 论述转引，2026-07-16）

要不要读代码？取决于任务能否被机器验证

「I read the code」论战 · 2026-07

Mitchell Hashimoto 发帖称自己读 Claude 写的代码，引发持续一周的论战——恰逢 Anthropic Fable 与 GPT-5.6 发布，「just vibe code it」派正觉得被新模型证实。

强硬派 · Cindy Sridharan

「每次听到有人说『Claude 全写的，我不懂原理』，我就认定他们 debug 不了——不能 debug 就不能 own；不能 own，在乎可靠性的人就无法把你当供应商信任。」

务实派 · Beyang Liu

Sourcegraph

「不必逐行读 AI 生成的代码，但 push 前应该把 diff 快速过一遍。」

立场差异的实质：信任边界画在人，还是画在验证机制。

0xCodez · Loop Engineering 判断法（全网 220 万+ 浏览）

建 Loop（自动化 agent 循环）之前，先问四个问题：

1. **任务是否重复？**一次性的活，好 prompt 更划算
2. **有没有东西能自动判定「干砸了」？**测试 / 类型检查 / linter / 构建至少一个；没有就得自己逐行读 diff，loop 就没省时间
3. **token 预算扛得住浪费吗？**loop 不产出也烧 token
4. **Agent 能跑自己写的代码吗？**有日志、能复现、看得到哪崩

附加题（最重要）：你打算 review 它产出的代码吗？**不打算，就别建 Loop。**

问题从「读不读」变成「能不能验证」——**机器可验证的任务才敢不看代码**，这正是 CH3 质量门控与 CH4 工程实践的主题。

来源：daily.dev「I read the code」论战复盘（2026-07-17）；0xCodez《Loop Engineering》（2026-06-21，Datawhale 转发）

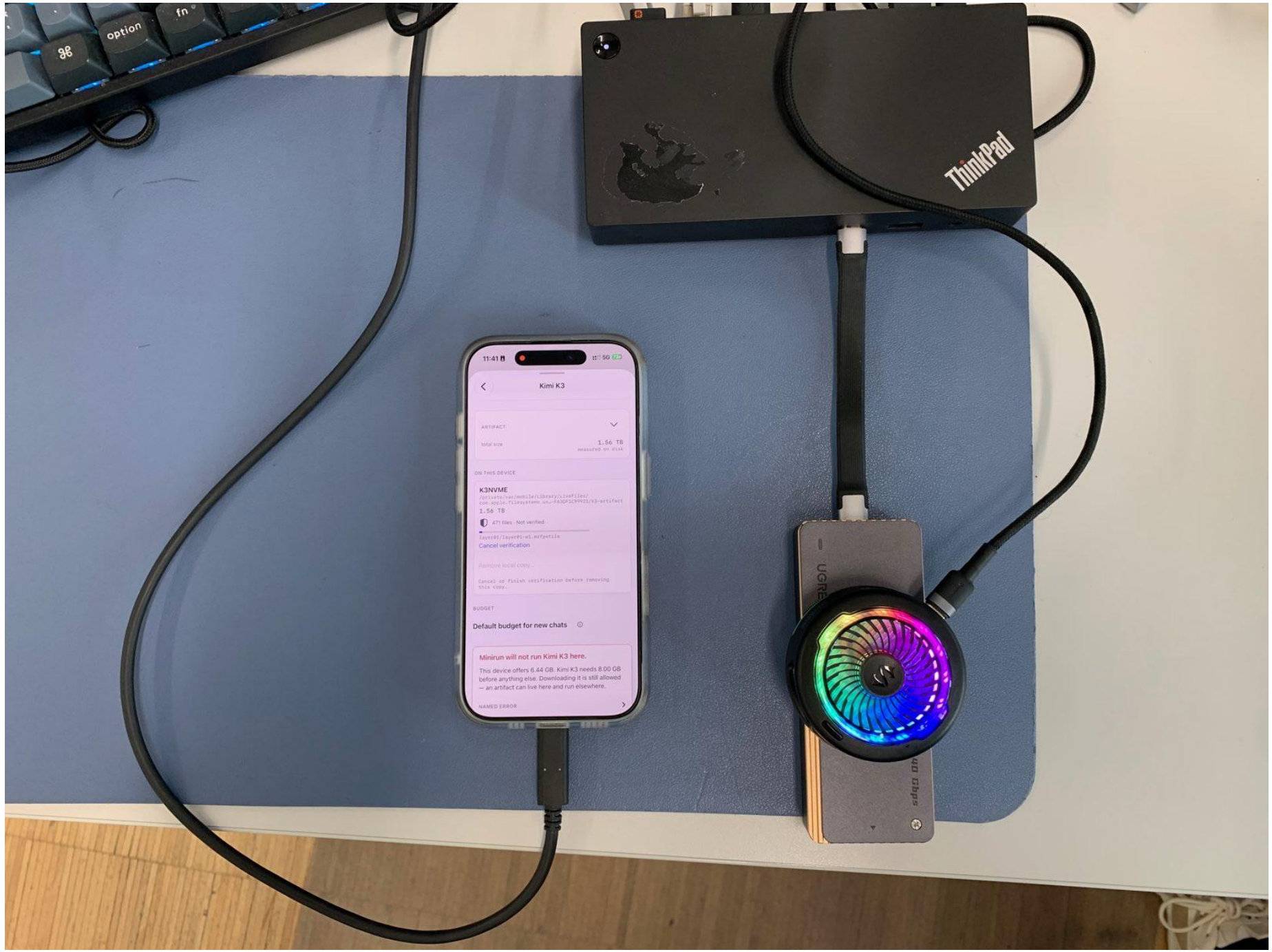
CH1 · PARADIGM SHIFT

01

K3 时代：开源大模型的范式跃迁

2.8T MoE + KDA 混合线性注意力 + Stable LatentMoE：K3 把开源模型推进「大参数、低激活」时代，也把端侧部署的「不可能三角」——能力、速度、资源不可兼得——摆到每个工程师面前。

P09 开源格局背景 · P10 K3 架构革新 · P11 端侧不可能三角 · P12 四重约束


option
fn
ThinkPad
11:41
Kimi K3
ARTIFACT
total size
1.56 TB
measured on disk
ON THIS DEVICE
K3NVME
1.56 TB
471 files · Not verified
Cancel verification
Remove local copy…
Cancel or finish verification before removing this copy.
BUDGET
Default budget for new chats
Minirun will not run Kimi K3 here.
This device offers 6.44 GB. Kimi K3 needs 8.00 GB before anything else. Downloading it is still allowed — an artifact can live here and run elsewhere.
NAMED ERROR

开源模型进入「大参数、稀疏激活、端侧可运行」时代

MoE 稀疏激活让「规模」与「成本」脱钩。总参数决定知识容量，每 token 激活参数决定单次推理的算力与带宽开销——两者不再绑定。

以 Kimi K3 为例：2.8T 总参的 MoE 模型，每 token 仅激活约 104B 参数，并提供 1M token 上下文。

端侧需求的三重驱动：隐私（数据不出设备）、成本（零按 token 计费）、延迟（离线可用、无网络往返）。

问题随之改变：不再是「能不能跑」，而是「跑多快、准多准」。

开放权重模型	总参 / 激活	上下文
Kimi K3（2026-07）	2.8T / 约 104B	1M
Kimi K2（2025-07）	1.04T / 32B	128K
Qwen3-235B（2025-04）	235B / 22B	128K
DeepSeek-V3 / R1	671B / 37B	128K
Llama 4 Scout（2025-04）	109B / 17B	10M

均为 MoE 稀疏激活架构；K3 激活参数另有「22B / 256 专家」的报道口径，数字以官方技术报告为准。

来源：interconnects.ai（K3，2026-07） · Kimi K2 技术报告（arXiv:2507.20534） · fireworks.ai（DeepSeek）

KDA + Attention Residuals + Stable LatentMoE：用稀疏化与线性注意力换 3T 规模

KDA 混合线性注意力

以线性注意力为主干替换全注意力的平方开销，长上下文计算量与 KV 占用随序列近线性增长，1M 上下文的前提。

Attention Residuals

为注意力输出保留跨层残差通路，缓解超深网络的梯度衰减，让更大层数下的训练保持稳定。

Stable LatentMoE

896 个专家、每 token 路由激活 16 个；用稀疏激活把 2.8T 总参的推理开销压到约 104B 激活的等效水平。

架构取向：不再堆稠密参数，而是用稀疏激活 + 线性注意力，把 3T 级规模拉回可推理区间。

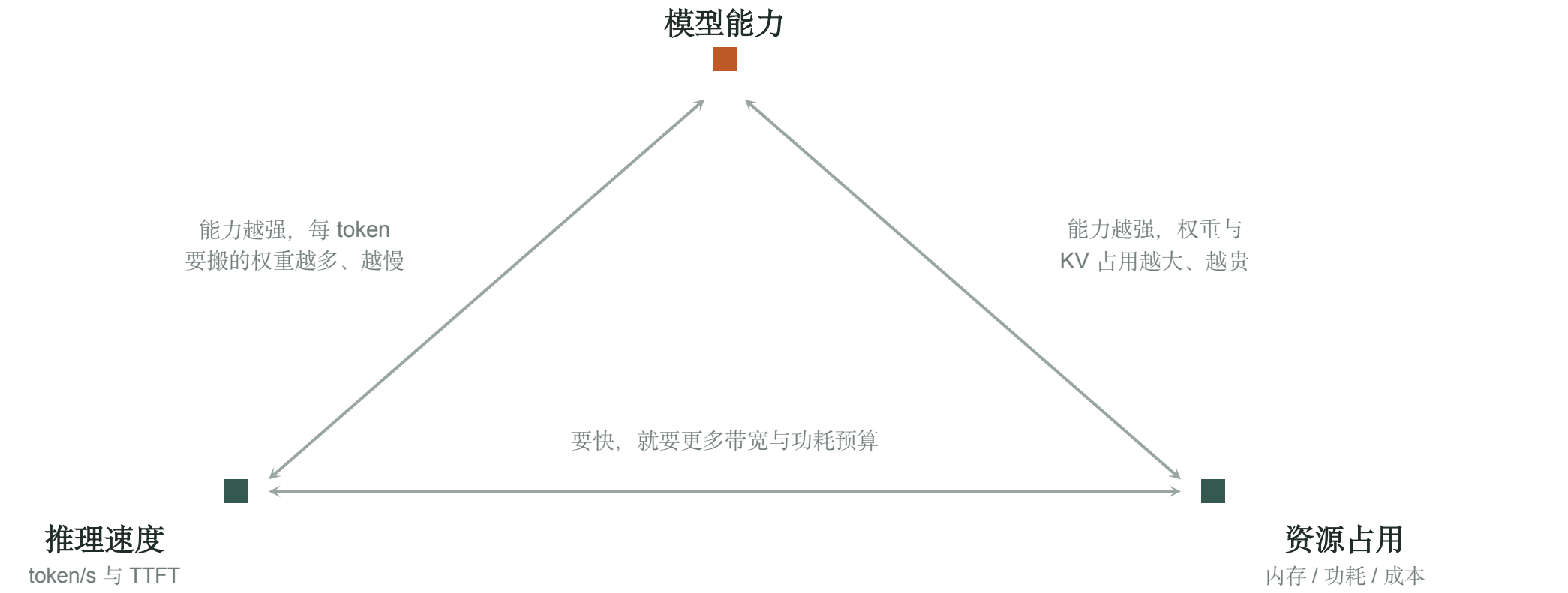
K3 关键参数事实

项目	数值
总参数	**2.8T**（MoE）
每 token 激活	**约 104B**
专家结构	896 专家 · 选 16
上下文	1M（1,048,576）token
模态	原生视觉理解

官方称相比 K2 扩展效率提升约 2.5 倍；2026-07-16 上线 API，2026-07-27 开源权重。

K3 架构与参数：Kimi K3 官方发布口径（2026-07），激活参数存在报道差异，以官方技术报告为准 · interconnects（2026-07-20）

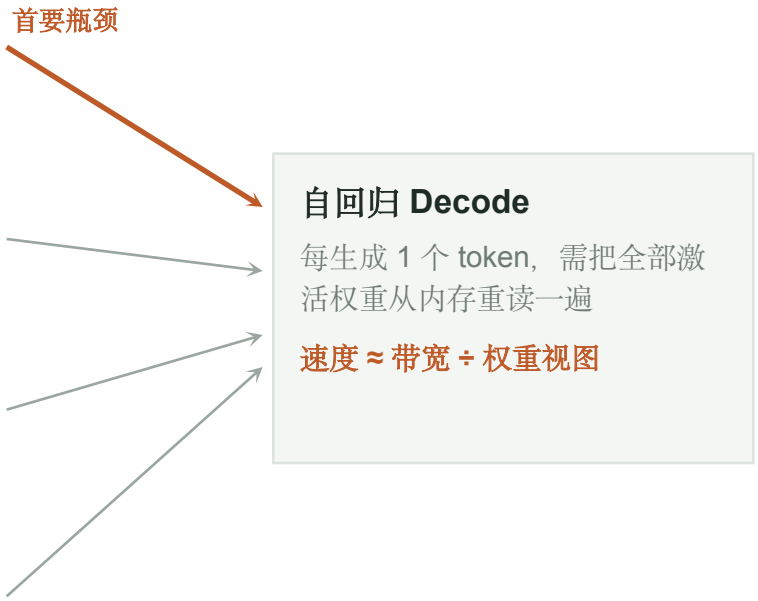
端侧部署的不可能三角：能力、速度、资源只能同时满足两个



破局思路： 压缩（CH2 量化 / 稀疏 / PD 分离）× 稀疏激活（MoE）× 端云协同（CH3 大小脑分工）——把三角的边整体往外推，而不是在内部取舍。

四重约束夹击端侧推理，内存带宽是首要瓶颈

- **内存带宽**
  权重搬运速度，decode 上限
- **算力（NPU TOPS）**
  决定 prefill 与并行计算
- **功耗与散热**
  被动散热限制持续性能
- **交互延迟**
  首 token 与逐字输出体验



60 GB/s
Apple A18 Pro 内存带宽（LPDDR5X，Apple 官方规格）

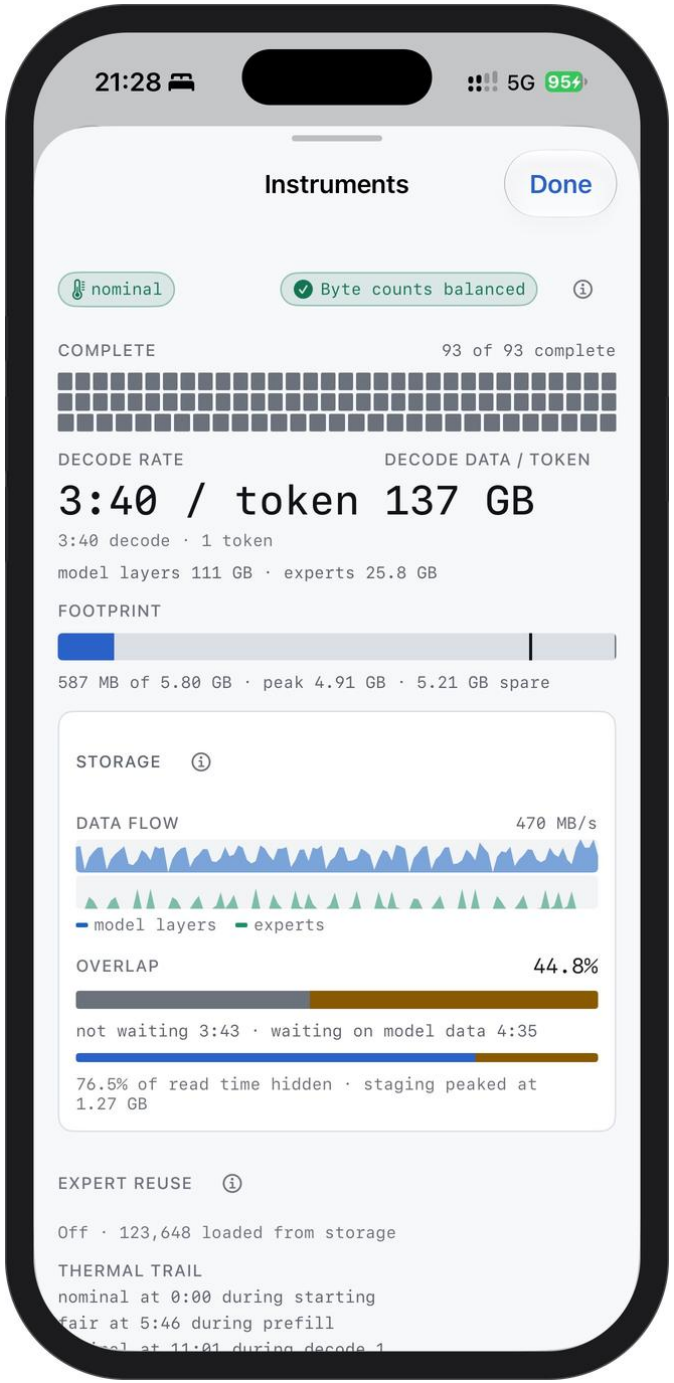
≈85 GB/s
旗舰 Android 平台 LPDDR5X 10.7Gbps 理论带宽（媒体口径）

35 / 45 TOPS
A18 Pro Neural Engine（官方）/ 骁龙 8 Elite（第三方口径，高通未公布）

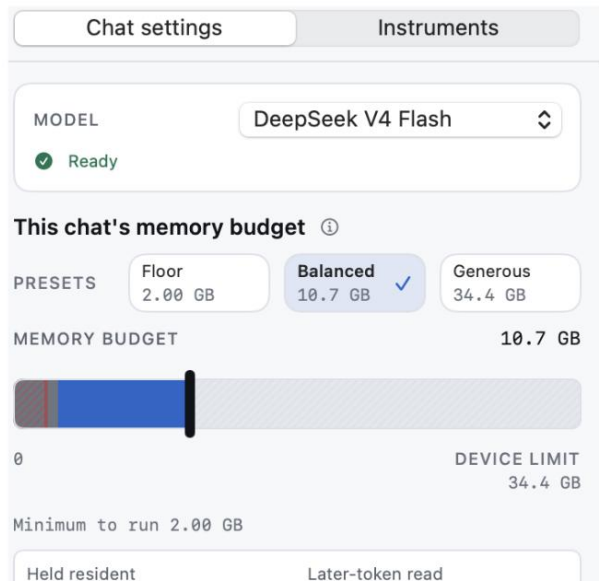
≈17 tok/s
7B INT4（约 3.5GB 权重）在 60 GB/s 下的 decode 理论上限（推算）

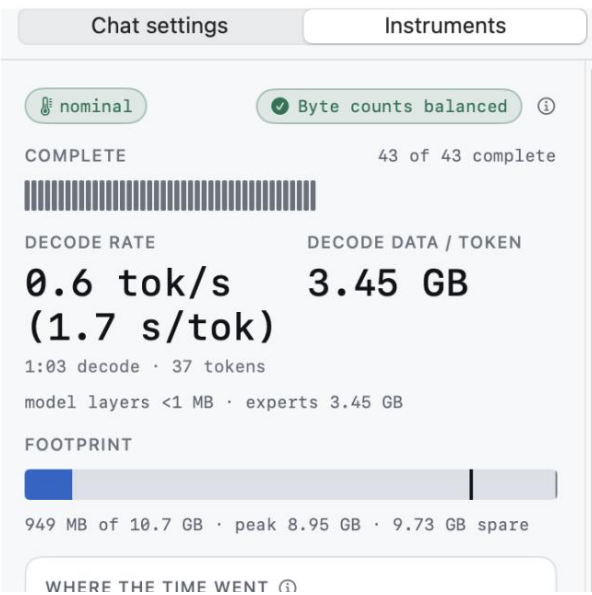
行业观察：40–50 TOPS、INT4/INT8 NPU 使 3–14B 模型端侧 20–100 tok/s 成为可行；微软 Copilot+ PC 以 40 TOPS 为门槛。

来源：Apple 技术规格（A18 Pro，60GB/s） · 36 氪：LPDDR5X 51→85GB/s（2026-08） · onoff.gr（NPU TOPS 汇总）



Model	On disk	Mac (M1 Pro, 32 GB)	iPhone 16 Pro	Status
DeepSeek V4 Flash — 284 B MoE	167 GB	≈ 1.7 s / token, 10.7 GB budget; runs at 2 GB	≈ 15 s / token, 3.8 GB budget; runs at 2 GB	Chat ready
Kimi K3 — 2.8 T MoE	1.56 TB	≈ 70 s / token, 8 GB budget	≈ 220 s / token, 5.8 GB budget, 2-token replies	Chat ready
DeepSeek V4.1 Flash — 552 B multimodal MoE	517 GB	≈ 5 s / token, 14.9 GB budget; runs at 3.4 GB	≈ 21 s / token, 1.9 GB budget	Chat ready
MiniMax H3 — audio & video generation	64 GB	download, verify and store	download, verify and store	Container ready
Qwen3.8-27B	—	—	—	Coming soon





CH2 · CORE TECH STACK

02

端侧推理性能优化的核心技术栈

从量化、KV Cache、Token 稀疏化、推测解码，到内核重写、MoE 专项与硬件适配，
逐层拆解端侧推理的性能杠杆，每条技术线都给出可核查的公开依据或工程实践口径。

P14 三层栈全景 · P15 INT4 量化 · P16 TurboQuant · P17 KV Cache · P18 Token 稀疏化 · P19 权重流式
P20 推测解码 · P21 内核与批处理 · P22 MoE 优化 · P23 硬件适配 · P24 引擎选型 · P25 实测 Benchmark

端侧优化是一个三层栈：模型压缩 × 运行时加速 × 硬件适配

L1 模型压缩
改模型本身

INT4 / INT8 量化 · 结构化剪枝 · 知识蒸馏

直接缩小权重体积与读取量，是收益确定性最高的一层，端侧部署从这里开始

→ P15 / P16
量化数据

↓ 产出：更小的模型（权重体积 ↓）

L2 运行时加速
改推理过程

KV Cache 压缩 · 推测解码 · 算子融合 / 图优化

压低长上下文的内存占用、把带宽瓶颈转化为并行度收益——本层决定「体验速度」

→ P17 / P18 / P20
KV Cache · 推测解码

↓ 产出：更低的每 token 时延与内存占用

L3 硬件适配
改执行映射

NPU / GPU / CPU 异构调度 · 推理引擎选型

把前两层的结果映射到具体 silicon：llama.cpp、MLC-LLM、ExecuTorch 等

→ P24 / P25
引擎对比 · 实测

三层自下而上叠加生效：压缩决定「装不装得下」，运行时决定「跑多快」，硬件适配决定「落在哪颗芯片上」。各层数据见后续对应页。

INT4 量化压缩约 70% 体积且精度近乎无损，是端侧部署的默认起点

-69.4%

GGUF Q4_K_M 体积缩减
（Llama-3.1-8B 统一评测）

精度持平：Q4_K_M GSM8K 77.41，vs FP16 的 77.63。

低于 4 bit 开始掉队：Q3_K_S 体积省得更多（-77.2%），但 GSM8K 降至 68.31。

趋势：Kimi K2 Thinking 已采用原生 INT4 量化训练，端侧 NPU 正向 INT4/INT8 数据通路收敛。

工程口径：默认 Q4_K_M，富余上 Q5_0

方案	位宽	体积缩减	精度 / 说明
FP16（基线）	16 bit	—	GSM8K 77.63
GGUF Q4_K_M	≈4.5 bit	**-69.4%**	GSM8K 77.41，与 FP16 基本持平
GGUF Q5_0	≈5 bit	-65%	评测无损，Pareto 推荐档位
GGUF Q3_K_S	≈3 bit	-77.2%	GSM8K 降至 68.31，精度明显受损
GPTQ	3/4 bit	—	逐层一次性量化，175B 级数小时完成
AWQ	4 bit	—	保护约 1% 显著权重，优于 RTN，可边缘部署

来源：GGUF K-quants 统一评测（arXiv:2601.14277，Llama-3.1-8B） · GPTQ（ICLR 2023） · AWQ（MLSys 2024） · K2 Thinking 原生 INT4

TurboQuant 离群值感知量化 + 结构化剪枝：4bit 精度损失压到 0.7% 以内

原理：离群值感知（outlier-aware）

按通道统计激活分布，把少量离群通道单独分组、保留更高位宽，其余通道统一压到 4bit；再配合结构化剪枝去掉低贡献通道，避免均匀量化被离群值拖垮。

<0.7%

TurboQuant 4bit 量化下的精度损失（相对 FP16 基线）
工程实践口径 · 内部测试环境

公开参照：GGUF Q4_K_M 在 Llama-3.1-8B 上 GSM8K 77.41 vs FP16 77.63，4bit 量化基本持平（arXiv 统一评测）。

三档低精度格式对比

格式	体积 vs FP16	精度损失	适用硬件
INT4	≈25%	<0.7%（TurboQuant，工程实践口径）	端侧 NPU / 消费级 GPU
FP8	≈50%	接近无损（W8A8 口径）	数据中心 GPU（H 系列）
MXFP4	≈25%	微缩放块浮点，接近 INT4 上界	昇腾 950（原生支持）

选型逻辑：端侧走 INT4 压体积，云侧 H 卡走 FP8 保精度，国产超节点走 MXFP4 吃原生算力。

4bit 损失 <0.7% 为工程实践口径 · 内部测试环境；公开参照：GGUF K-quants 统一评测（2026-01） · AWQ（MLSys 2024）

长上下文时代，KV Cache 取代权重成为内存大头，压缩决定可用上限

注意力变体：KV 头数直接决定缓存体积

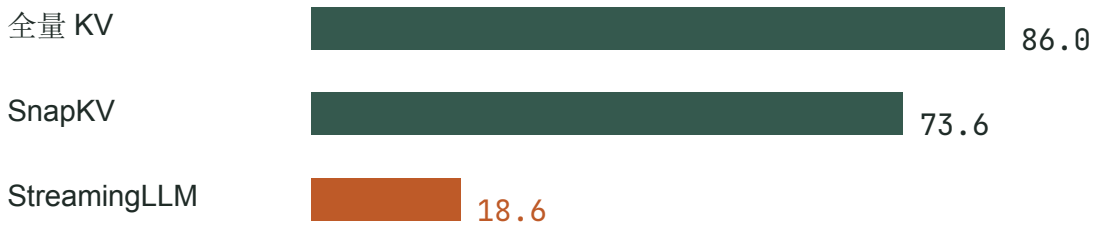


每个方块 = 一组 KV 头；缓存体积随序列长线性增长，128K 级上下文下可超过模型权重本身。

GQA（Google，EMNLP 2023）以接近 MQA 的速度保持接近 MHA 的质量，已成为 Llama / Qwen 等主流模型的默认配置。

结构省不下的部分，只能压缩——而压缩不是免费的。

10% KV 保留率下的 RULER 均分（Llama-3.1-8B）



同样的保留率，方法之间长上下文能力差距可达数倍

SnapStream：prefill 用 SnapKV，decode 用 StreamingLLM

DeepSeek-R1 671B（TP16）部署下：KV 显存降 **4×**、解码吞吐升 **4.3×**，prefill 延迟增加 ≤5%。

压缩方法对任务的敏感度差异巨大：端侧长上下文必须按目标任务实测，不能直接照搬论文保留率。

来源：GQA（arXiv:2305.13245） · KV 压缩评测（arXiv:2502.01068，RULER） · SnapStream（arXiv:2511.03092）

动态 Token 稀疏化 + KV 分层存储：砍掉 40% 无效缓存

机制：注意力得分动态筛除低贡献 token

每一层 decode 时按注意力得分给历史 token 排序，低于动态阈值的 KV 不再参与本轮计算；长上下文下大部分历史 token 贡献趋近于零，筛除后精度几乎不变。

注意力得分 → 动态阈值筛选 → 保留高贡献 token

-40%

无效 KV 缓存削减幅度（1M 上下文长文档场景）
工程实践口径 · 内部测试环境

KV 三层存储：热 / 温 / 冷分级



分层依据是访问频率而非语序：稀疏化先砍掉低贡献 token，分层存储再压低留存部分的驻留成本。

-40% 为工程实践口径 · 内部测试环境；公开参照：SnapStream（SnapKV + StreamingLLM，KV 显存降 4×）

权重流式加载：模型比内存大，就从 SSD 逐层流进来

带宽预算决定速度

显存 / 内存（预算制）
预算 5.8–10.7 GB · 固定缓冲区

外置 NVMe SSD · 权重全量驻留
Kimi K3 1.56 TB（需 2 TB 盘）· DeepSeek V4 Flash 167 GB

逐层读入 + MoE 仅读路由命中的专家 · 零拷贝交给 MLX 固定缓冲区 · 用完释放 | 瓶颈在硬盘盒与线缆而非 SSD：Mac 用 USB4/Thunderbolt 直插，iPhone 需挂供电坞

实测：开源项目 Minirun 在两台消费设备上跑超大模型

模型	体积	Mac M1 Pro 32GB	iPhone 16 Pro	状态
Kimi K3（2.8T MoE）	1.56 TB	≈70 s/token @ 8 GB 预算	≈220 s/token @ 5.8 GB 预算	Chat ready
DeepSeek V4 Flash（284B MoE）	167 GB	≈1.7 s/token @ 10.7 GB（2 GB 也能跑）	≈15 s/token @ 3.8 GB（2 GB 也能跑）	Chat ready

项目 README 口径 · 参考设备见来源，速度随硬盘/线缆/预算变化

① **预算制内存：**设定即上限，拒绝优于超限，任何预算下输出字节一致。

② **供应链验证：**对 HF 已发布树摘要校验 + 描述符白名单，数据不出设备。

③ **内置 Instruments：**decode 速率、bytes/token、占用 vs 预算实时可见。

收束：内存墙 → 带宽预算问题——3T 级模型在消费设备上可运行、可验证、可审计；速度不足以对话，适合离线批处理与研究。

与 P17 KV 分层、P18 Token 稀疏同属内存层次优化（作用于权重）；隐私立场呼应 P29。

来源：nanguoyu/minirun-app（Apache-2.0 · Swift/MLX · v0.4 · 作者 Dong Wang / TU Graz）

推测解码把内存带宽瓶颈转化为并行度收益



为什么有效

decode 阶段受内存带宽限制：每生成 1 个 token 都要把权重完整读一遍。让小模型串行起草、大模型一次前向并行验证，读一遍权重可确认多个 token，串行解码由此变成批量验证。

接受率是关键变量：同族草稿-目标配对接受率 55–82%；batch=1 典型加速 2–4×，batch>16 不建议使用。

来源：推测解码综述 arXiv:2601.05149（Medusa/EAGLE 数字为转引）；生产口径 localaimaster.com

2–3×	原始方案 Leviathan et al.：小模型起草、大模型并行验证，输出无损
≤3.6×	Medusa 多头并行起草（综述转引，原论文 arXiv:2401.10774）
≤4.3×	EAGLE-2 动态起草树（综述转引，原论文 arXiv:2401.15077）

FlashKDA 内核重写 + 算子融合 + 连续批处理：GPU 利用率做到 85% 以上

01 FlashKDA 内核重写

机理：为 KDA 线性注意力重写 CUDA 内核，分块状态计算 + 合并显存访问。

收益：长序列下 attention 耗时显著下降，1M 上下文可用。

02 算子融合

机理：RMSNorm / RoPE / SwiGLU 等逐元素算子融合进主内核，减少 kernel launch 与中间显存读写。

收益：调度开销与显存往返同步下降，小 batch 也吃得满。

03 连续批处理（Continuous Batching）

机理：按请求粒度动态拼批——完成即退出、新请求即时插入，不等整批收尾。

收益：消除气泡空转，是利用率冲上 85% 的直接来源。

三者叠乘：内核决定单步算得多快，融合决定开销多小，批处理决定空转多少——分别压住计算、调度、利用率三条线。

85%+

生产环境 GPU 利用率

工程实践口径 · 内部测试环境

连续批处理示意（请求进出动态拼批）



GPU 利用率 85%+ 为工程实践口径 · 内部测试环境

MoE 专项三件套：EP 专家并行、AF 分离、PD 分离

PD 分离：Prefill 与 Decode 分池部署

Prefill 节点组

算力密集（compute-bound）

整段 prompt 并行计算，优先堆 FLOPs 与大 batch

KV Cache 传输 →

Decode 节点组

带宽密集（memory-bound）

逐 token 生成，优先堆内存带宽与并发路数

拆开之后才能独立扩缩容：P 侧按 prompt 长度配算力、D 侧按并发配带宽，两个阶段不再互相拖累，同预算下 TTFT 与吞吐同时改善。

引擎适配：vLLM Ascend 与 SGLang 均已提供 PD 分离 + EP 的部署模式，K3 的 896 专家结构直接复用该路径，无需自研调度层。

EP 专家并行

896 个专家按路由热度切分到多卡；路由层加负载均衡约束，防止热门专家堆积、冷门专家闲置。

AF 分离（Attention–FFN）

Attention 与 FFN 拆到不同流水级：Attention 吃带宽、FFN（MoE）吃算力与通信，分别匹配资源特征。

组合关系

EP 解决「专家放哪」，AF 解决「层内怎么切」，PD 解决「阶段怎么分」——三层正交，可同时启用。

部署模式为 vLLM / SGLang 公开特性；K3 专家数按 Kimi K3 官方发布口径（2026-07），以官方技术报告为准

硬件适配三分天下：昇腾超节点、高通 AI Engine、联发科天玑 APU

平台	定位	关键特性	适配要点
昇腾 950 超节点	云侧集群推理	原生支持 MXFP4 微缩放量化；超节点内高带宽互联，面向 K3 级 MoE 的 EP / PD 分离部署	走 vLLM Ascend 后端，量化档选 MXFP4 吃原生算力
高通 AI Engine	手机端 SoC	Hexagon NPU 统一调度 CPU / GPU / NPU；llama.cpp 的 Hexagon 后端在推进中	端侧 3–14B 走 INT4 + NPU 卸载，兼顾功耗墙
联发科天玑 APU	手机端 SoC	天玑旗舰平台集成 APU，面向端侧生成式 AI 场景开放调度接口	与高通同档 INT4 路线，按机型实测带宽定量化档位

选型原则： 云侧 K3 级模型押注昇腾超节点 + MXFP4；端侧小模型在高通 / 联发科平台走 NPU 调度，量化格式跟着硬件原生支持走，不逆着芯片做优化。

硬件细节：厂商资料与第三方口径整理；Hexagon 后端进展见 llama.cpp（GitHub）

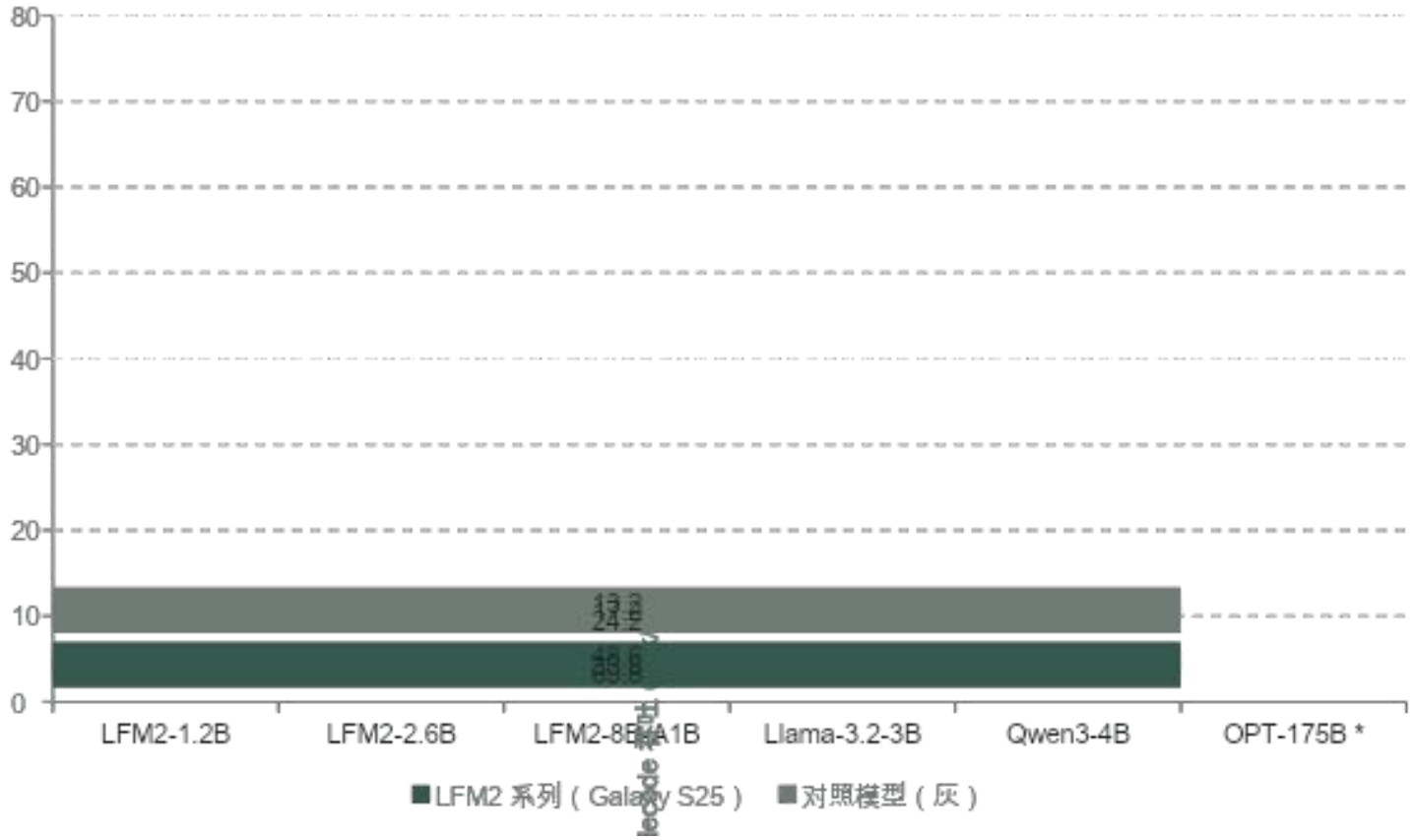
引擎选型取决于平台覆盖与量化格式生态，没有全优解

引擎	平台覆盖	后端与量化	特色定位
llama.cpp	桌面/移动全平台；Metal / CUDA / Vulkan / SYCL / OpenCL（Adreno），Hexagon NPU 后端进行中	GGUF 格式，1.5–8 bit 整数 K-quants，CPU+GPU 混合推理	纯 C/C++ 零依赖（MIT）；GGUF 已成端侧量化格式事实标准
MLC-LLM	手机、桌面、浏览器（WebGPU / WebLLM）	自研编译栈按硬件 codegen，跨设备复用	可在浏览器内运行，也可起本地 OpenAI 兼容服务
ExecuTorch	iOS / Android / Linux / Windows	XNNPACK、CoreML、MPS、Vulkan 后端	PyTorch 官方移动/边缘运行时，与训练栈同源，导出链路最短
ONNX Runtime GenAI	跨平台 CPU / GPU / NPU	ONNX 图 + DirectML / CUDA 等执行提供程序	微软系工程化部署，模型即 ONNX 图，适合 Windows/Copilot+ 生态
PowerInfer	消费级 GPU + CPU 混合（单机）	利用神经元激活幂律稀疏性，热神经元驻留 GPU、冷神经元走 CPU	SJTU（SOSP'24）：单张 RTX 4090 跑 175B 级模型平均 13.20 tok/s，比 llama.cpp 快至 11.69×

选型判断：要生态与格式选 **llama.cpp**（GGUF）；要浏览器选 **MLC-LLM**；要 PyTorch 链路选 **ExecuTorch**；要 Windows/NPU 管线选 **ONNX Runtime GenAI**；要消费级单卡跑超大模型选 **PowerInfer**。

来源：llama.cpp GitHub；PowerInfer（SOSP'24，arXiv:2312.12456）；awesome-edge-ml-models

旗舰手机跑 1B 级模型已达 70 tok/s，端侧可用性取决于模型规模



测试条件

手机组（前 5 根柱）：Galaxy S25（Snapdragon 8 Elite），llama.cpp CPU 后端，Q4_0 量化，batch=1，1K token 前缀续写 100 token，取 decode 吞吐。

*** OPT-175B（末柱）**：单张 RTX 4090，PowerInfer（GPU-CPU 混合），平均 13.20 tok/s、峰值 29.08 tok/s。

读法：旗舰手机跑 1–3B 模型已达 25–70 tok/s，体验可用；175B 级模型即便桌面级 GPU 也只有约 13 tok/s。端侧可用的前提是模型够小、量化够狠。

来源：LFM2 Technical Report（arXiv:2511.23404，Galaxy S25 / llama.cpp Q4_0 / batch=1）；PowerInfer（arXiv:2312.12456，SOSP'24）

03

CH3 · AGENT COLLABORATION

端侧 Agents 协作架构设计

单 Agent 触顶之后，问题变成如何分工：端侧小脑 + 云端大脑，ReAct / Handoff 让协作可审计，
端云动态卸载按任务复杂度、延迟敏感度与隐私约束，决定每一步留在端侧还是上云。

P27 为什么协作 · P28 小脑 + 大脑 · P29 安全论据 · P30 双因子 · P31 协作模式 · P32 ReAct & Handoff · P33 可靠性循环 · P34 AI-Native SDLC
P35 制品链 · P36 协议 · P37 调度 · P38 质量门控 · P39 端云拓扑 · P40 小模型角色 · P41 挑战

失败率 41%–86.7%：争论焦点是协作的结构，而非要不要协作

实证方：MAST 多智能体失败分类法（UC Berkeley 等）

41–86.7%

7 个主流多智能体框架的任务失败率区间

· 分析 1,600+ 条 trace（标注一致性 κ=0.88），归纳 3 大类 14 种失败模式

· 规格与系统设计缺陷 ≈41.8%；智能体间失合约 ≈36.9%；任务验证缺失 ≈21.3%

· 修补性干预（改 prompt、换模型）仅带来 +14% 任务完成率

· 结论：需要结构性重设计，而非局部调优

反方：Cognition《Don't Build Multi-Agents》（2025-06）

· 并行子智能体各自持有割裂的上下文，局部决策互相冲突（Flappy Bird 构建案例）

· 协调开销随智能体数量失控，调试与归因几乎不可行

· 主张：单线程主智能体 + 上下文工程；把子任务当「工具」调用，而非对等 Agent

隐含判断：不可靠的不是分工本身，而是无共享上下文、无验证的并行分工。

工程读法：双方不矛盾——单 Agent 受上下文与可靠性约束必须分工，但分工必须带**共享上下文**与**验证门**，这正是 P31 与 P33 的主题。

来源：MAST（arXiv:2503.13657，NeurIPS 2025 D&B）；Cognition《Don't Build Multi-Agents》（Walden Yan，2025-06-12）

端侧小脑 + 云端大脑：1B–7B 管感知与路由，K3 管深度推理

−40~60%
Token 成本下降
工程实践口径 · 内部测试环境

端侧 · 设备信任域（数据不出设备）

端侧小脑 · 1B–7B 轻量模型
实时感知 · 快速决策 · 隐私过滤
常驻 NPU · 毫秒级 · 零 token 计费

默认路径：简单与敏感请求在端侧闭环

承担角色
· 意图识别与路由
· 敏感信息脱敏
· 简单任务直接作答

脱敏请求 →

← 结构化结果

云端 · 信任边界（仅承接脱敏请求）

云端大脑 · Kimi K3
深度推理 / 长程规划 / 复杂编排
2.8T MoE · 按 token 计费

承接端侧判定为「复杂」的请求，返回结构化结果

动态卸载决策带：三维判定 → 留端 / 上云

任务复杂度
高 → 倾向云端

延迟敏感度
高 → 留在端侧

隐私约束
敏感 → 留在端侧

路由判定
Router 策略

→ **留端**：实时 / 敏感任务，零 token 计费

→ **上云**：复杂推理交由 K3 深度生成

公开参照：Apple Intelligence 以约 3B 端侧模型 + 云端 PCC 大模型落地同构架构；第三代 AFM 的 Core Advanced 为 20B 稀疏端侧模型，按请求仅激活 1–4B 参数。

来源：Apple Foundation Models（arXiv:2407.21075）、Apple 第三代 AFM（2026-06）；−40~60% 为工程实践口径 · 内部测试环境

端侧模型的第二论据：不只是省 token，更是隐私、安全与自我主权

Vitalik 的威胁清单：agent 生态「极其漫不经心」

OpenClaw agent 无需人工确认即可改关键设置；恶意网页可劫持 agent 实例（HiddenLayer 演示）；约 15% 被审查 skills 含恶意指令。

01 **远程模型滥用隐私数据**：隐私数据被远程模型获取并可能被滥用

02 **LLM 越狱**：远程恶意内容「黑掉」agent，使其违背用户利益（转走币、外发隐私）

03 **LLM 失误**：把隐私发错渠道

04 **LLM 后门**：「开放权重 ≠ 开源」，模型可能被训入触发式后门

05 **第三方软件供应链**

他的防御原则：local-first + 一切沙箱化

· local-first 推理，文件本地托管

· 一切沙箱化：bubblewrap 按目录隔离文件与端口

· 本地知识库（1TB Wikipedia dump）减少搜索泄露

硬件实测 · Qwen3.5:35B（llama-server）

设备	生成速度
RTX 5090 笔记本	**90 tok/s**
AMD Ryzen AI Max Pro（128GB 统一内存）	51 tok/s
DGX Spark	60 tok/s

他的个人体验阈值：「低于 50 tok/s 就烦到不值得用」

「我们刚靠端到端加密和本地优先软件在隐私上前进一步，却眼看要因为把全部生活喂给云端 AI 而后退十步。」—— Vitalik Buterin（译自原文）

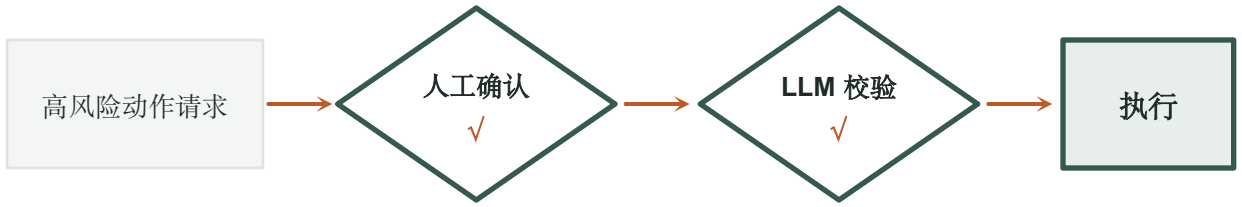
来源：Vitalik Buterin《My self-sovereign / local / private / secure LLM setup》（2026-04-02）

人机双因子：人 + LLM 的 2-of-2 确认，比单独信任何一方都安全

"the new 'two-factor confirmation' is that the two factors are the human and the LLM." —— Vitalik Buterin

意译：新的「双因子确认」，两个因子分别是人和 LLM——人会走神、会被骗；LLM 会犯错、会被针对性攻击；两者失败模式不同。

2-of-2 确认门：动作通过需双方放行



高风险动作的人工覆盖 = 更高摩擦 + 时延（故意设计，不是缺陷）

确认防火墙实例：他的 messaging-daemon

· agent 可自主做的只有两件事：**读消息、发消息给自己**

· 发给他人必须人工确认：本地 localhost 确认页，Approve / Deny

· 测试中 LLM 自行识别出诈骗邮件并拒绝 / 警告；但未来攻击会更复杂，人工确认不可省

· 以太坊钱包同理：每日小额（如 $100）免确认，同时限制 calldata / 金额 / 交易数

与 P28 大小脑的衔接：消毒路由

· **input sanitization**：本地模型在请求上云前剥离隐私数据——即 P28 的「脱敏请求」

· 愿景：顶层任务都由本地模型完成，由它判断何时呼叫更强的远程模型、问什么问题才能泄露最少信息

必须上云时：远端多层防御

· **ZK-API**：请求不可关联，服务器不知发送者身份

· **Mixnets**：IP 不可关联

· **TEE 推理**：验证 attestation，减少泄露但非密码学级安全

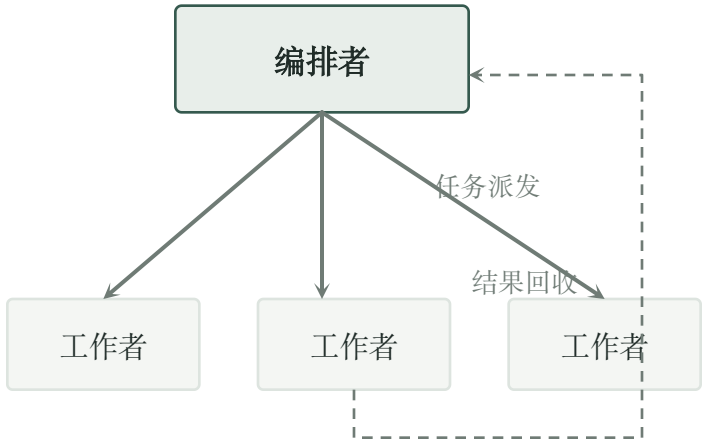
· 长期方向：**FHE**（全同态加密）

收束：**端云协同不只是成本问题，更是信任架构问题。**

来源：Vitalik Buterin《My self-sovereign / local / private / secure LLM setup》（2026-04-02）

三种协作拓扑：层级编排控制力最强，也是当前最稳的形态

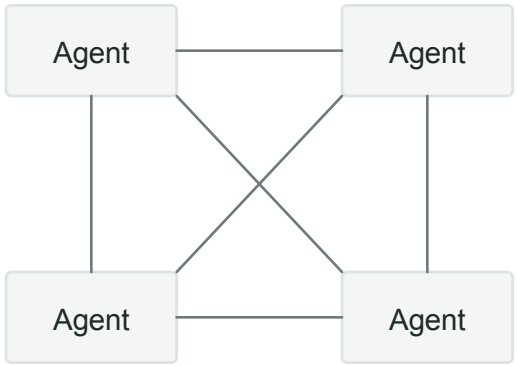
① 层级编排 Orchestrator-Worker 当前最稳



适用：任务可分解、需要质量门禁。

代价：编排者是单点故障与吞吐瓶颈。

② 对等 Swarm

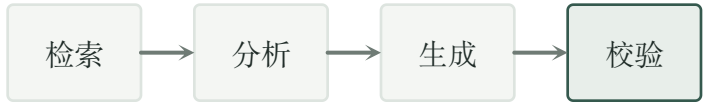


点对点协商，无中心节点

适用：探索性任务、角色平等。

代价：状态一致性与协调开销高，是 MAST 失败率高发区。

③ 流水线 Pipeline



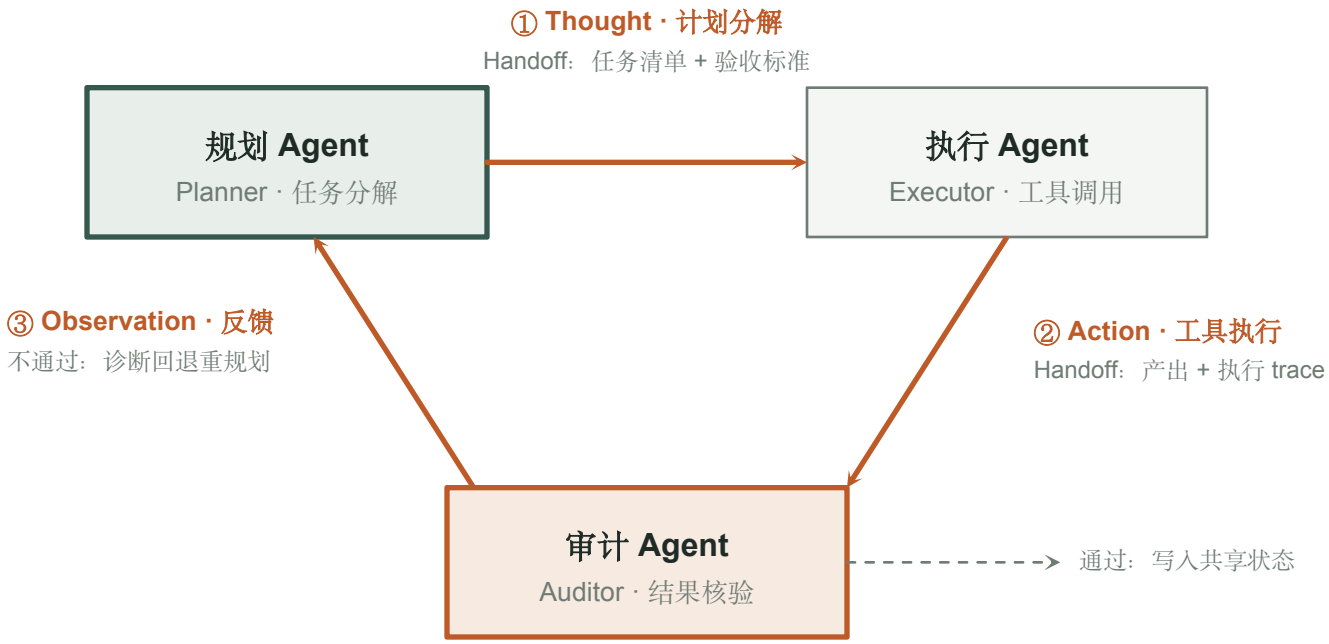
上一阶段产出即下一阶段输入

适用：流程固定的生产管线。

代价：无法回溯，上游失败沿链路级联放大。

来源：Agent 框架对比综述（2026-08）；失败率证据见 MAST（arXiv:2503.13657）

规划 → 执行 → 审计：ReAct 循环 + Handoff 让三角色闭环协作



ReAct 内环：单个 Agent 内部按 Thought → Action → Observation 迭代；
Handoff 外环：角色之间只移交结构化产物，每次移交都是一次审计点。

状态共享与记忆管理三件套

① 共享任务列表

单一事实来源：所有角色读写同一份任务状态，避免上下文漂移。

② 上下文窗口隔离

各角色只加载与职责相关的上下文切片，控制 token 开销。

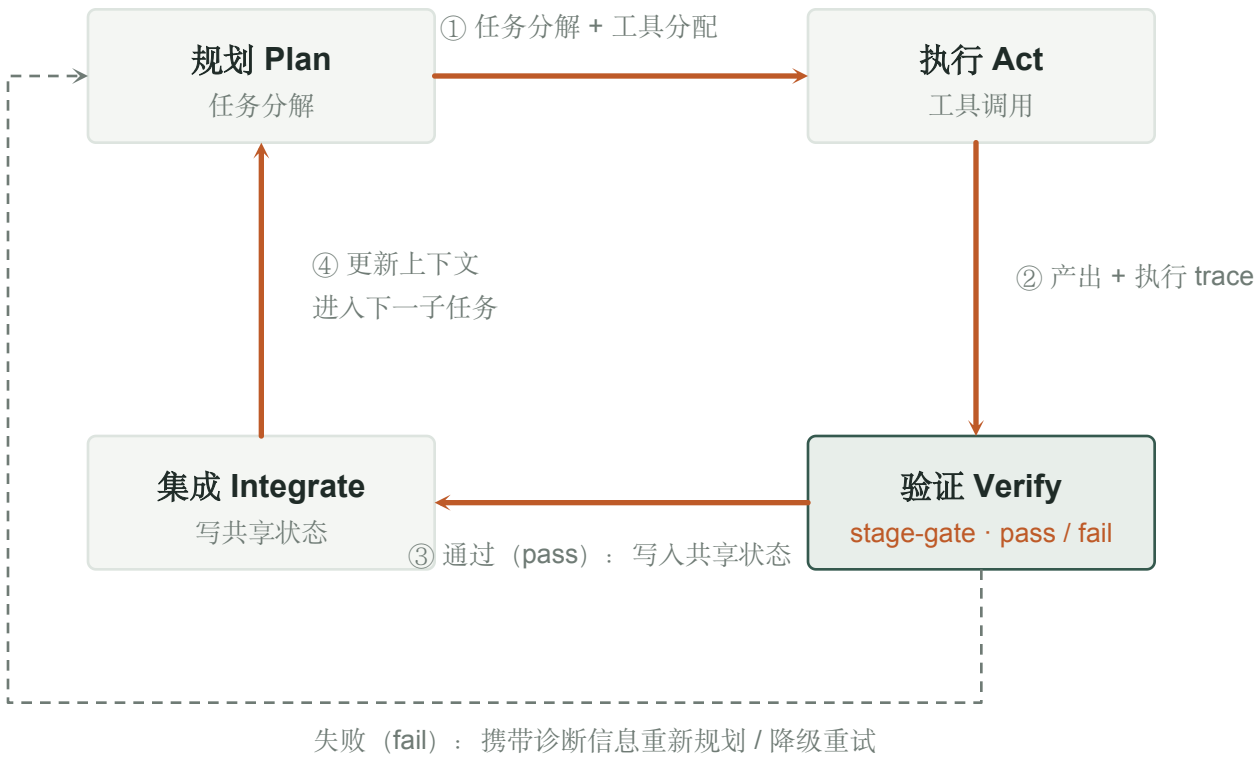
③ 自动摘要策略

长程执行 trace 滚动摘要，只保留关键决策与结论。

OpenAI Agents SDK 已将 Handoffs 列为与 Agents / Guardrails / Tracing 并列的核心原语。

来源：AI Agent 框架对比（Agents SDK 核心原语，2026-08）；ReAct / Handoff 机制为工程综述性总结

可靠 Agent 系统靠「规划-执行-验证-集成」循环，而非更长 prompt



阶段门（stage-gate）规则

· 每个阶段有明确的进入 / 退出条件，产出物可验证才放行

· 验证是二元门控（pass / fail）；fail 必须携带诊断信息回退，不允许「带病」进入集成

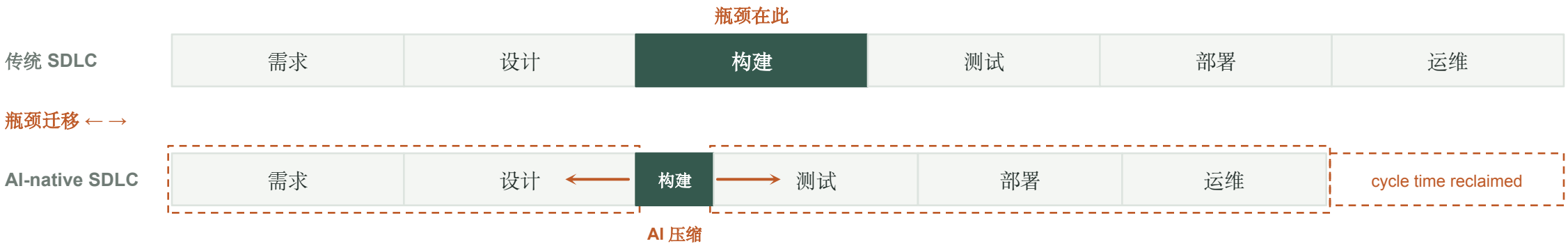
· 集成是唯一写共享状态的位置，避免并发写冲突

· 可靠性来自循环结构，不来自更长的 prompt

证据锚点：MAST 失败分类法中，任务验证缺失约占全部失败的 21.3%——验证门正是针对这一最高杠杆点。

来源：失败模式分布引自 MAST（arXiv:2503.13657，NeurIPS 2025 D&B）；循环结构为工程综述性总结

代码不再是瓶颈：瓶颈向构建环节的左右两侧迁移



三个后果

① 瓶颈从写代码迁移到计划、评审、部署

② 既有流程与控制点为「人写代码」设计，与 AI 生产速度错配

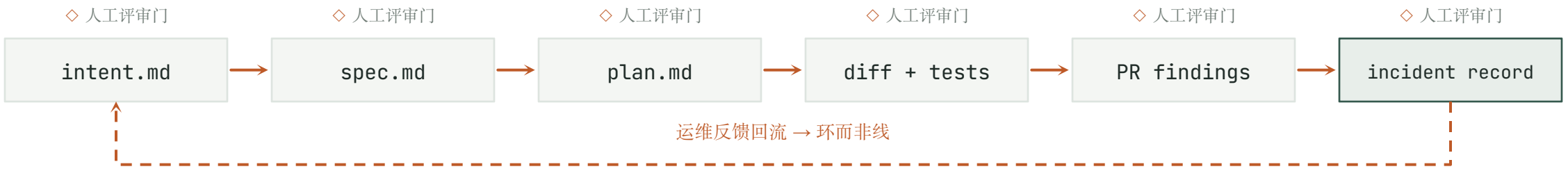
③ 治理与协调成本上升

例证：安全评审队列

代码产出速度翻数倍，安全评审人力不变——评审队列成为新的阻塞点，是瓶颈右移的典型例证。

来源：The AI-Native SDLC Playbook · Anthropic

AI-Native SDLC：制品链串起六阶段，人站在环上把门



两条原则

① *"Every stage commits an artifact the next stage can read"*

制品入库（git），整条 git 链即审计轨迹（audit trail）

② 人站在环上（humans above the loop）

发起（instigate）、引导（direct）、治理（govern），而非在环内逐步执行

生产门禁原则

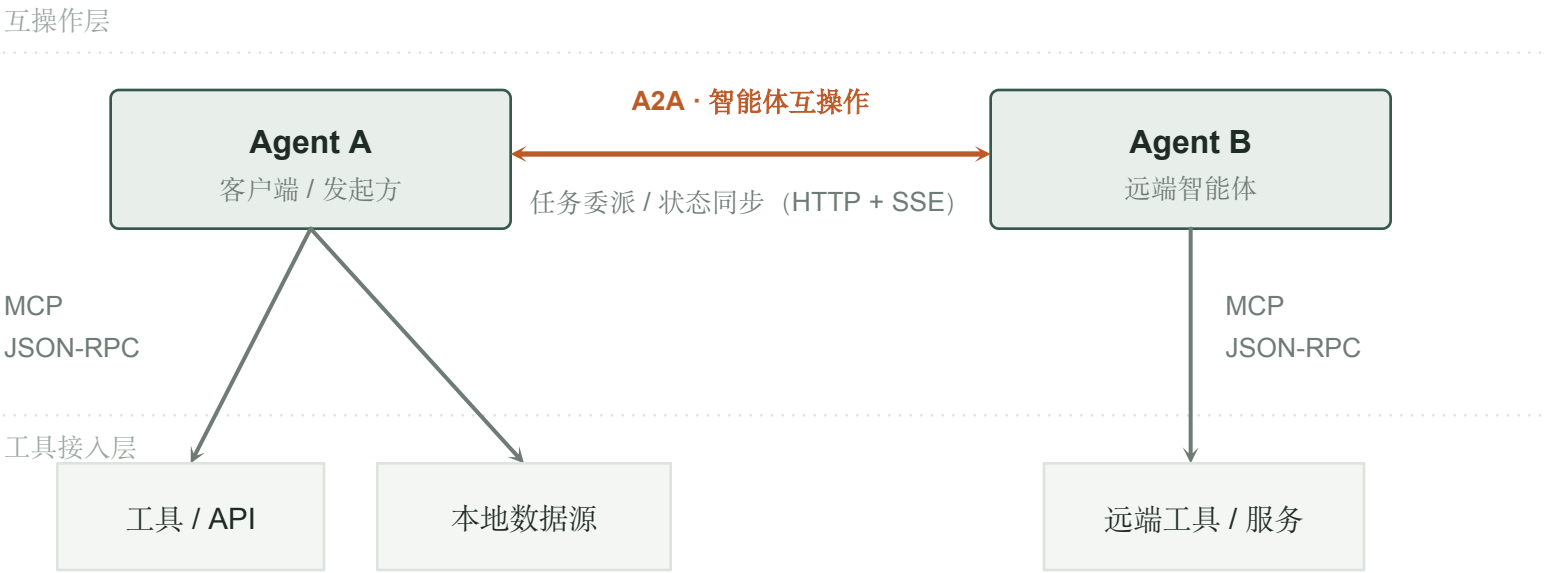
"The agent may act up to the production gate, and cannot pass it"

用 hooks 等机制强制；evals 嵌入 CI，随每次变更持续运行，而非发布前一次性测试。

收束：与 P38 质量门、P07 0xCodez 机器验证呼应——**制品链让每个环节都可机器校验、可人工追责。**

来源：The AI-Native SDLC Playbook · Anthropic

MCP 连接工具、A2A 连接智能体：通用接口正在分层成型



一个 Agent 同时是两层的使用者：对上通过 A2A 与其他 Agent 互操作，对下通过 MCP 接入工具与数据——协议各管一段，互不重叠。

MCP — 工具接入层

Anthropic 2024-11-25 发布；客户端-服务端 + JSON-RPC，被称为「AI 的 USB-C」；2025 年获 OpenAI、Google DeepMind、Microsoft 采纳。

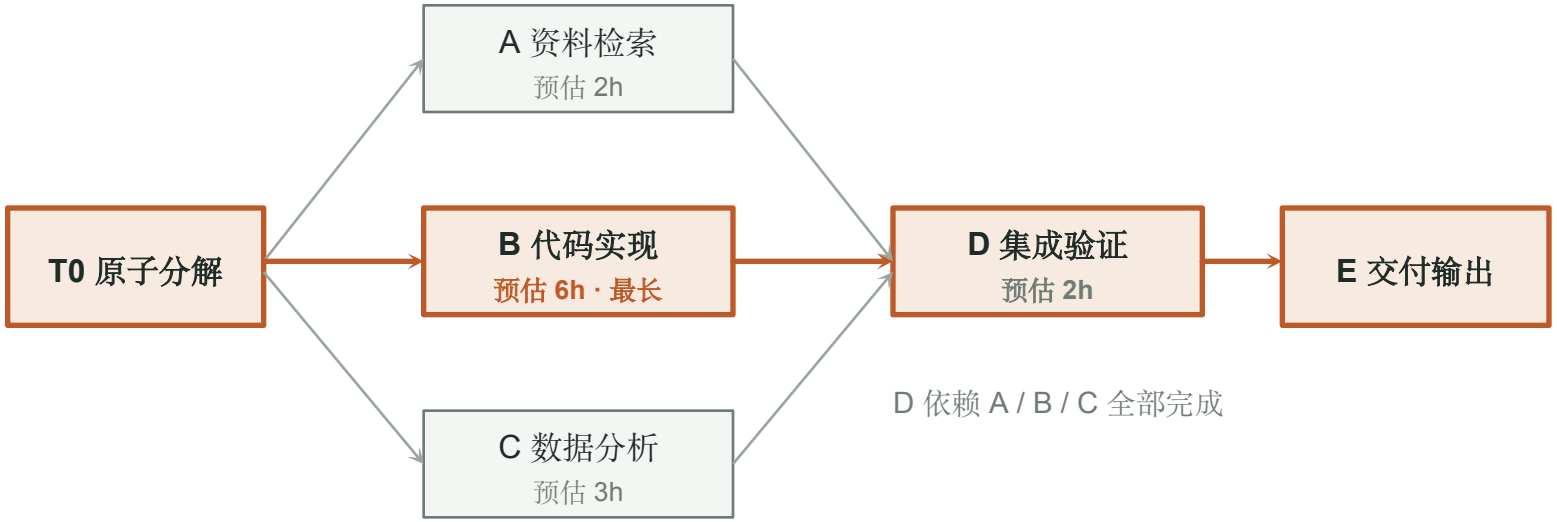
A2A — 智能体互操作层

Google 2025-04-09 于 Cloud Next 发布，50+ 合作伙伴；2025-06-23 捐赠 Linux Foundation，Apache 2.0；定位与 MCP 互补。

判断：组织内工具调用走 MCP；跨组织、跨厂商的 Agent 协作走 A2A。

来源：Anthropic MCP 公告（2024-11-25）；A2A 标准沿革（含捐赠 Linux Foundation，2025-06-23）

并行收益受任务依赖图约束，原子化分解是调度前提



- 关键路径 T0 → B → D → E：总时延由最长依赖链决定，给 A / C 增加并行 worker 无法缩短交付时间
- 可并行分支：A、B、C 之间无依赖，由调度器并发派发

原子化分解是前提

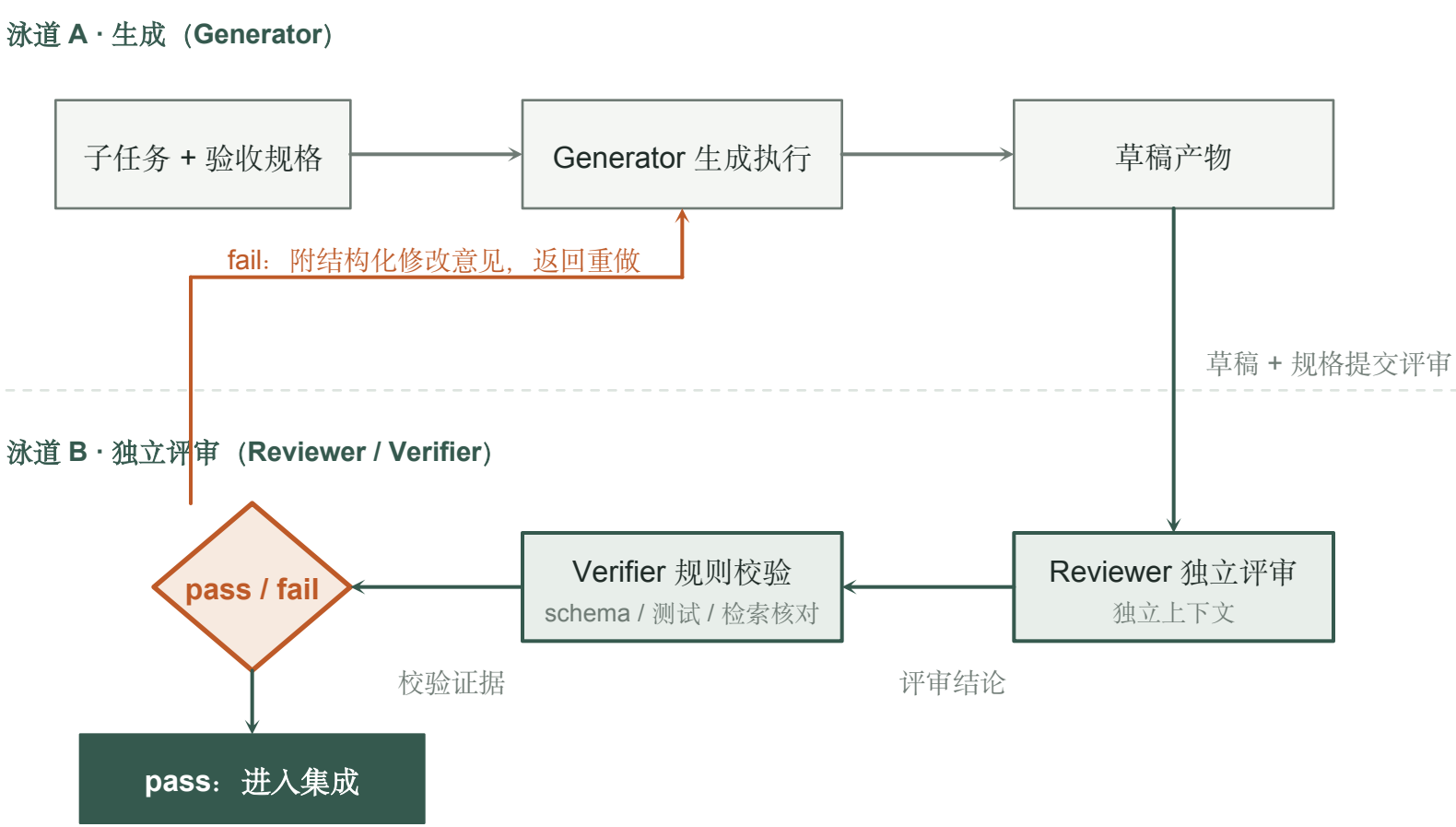
子任务粒度 = 一次工具调用 + 一个可验证产物。粒度过粗无法并行；过细则协调开销吞掉收益（MAST：智能体间失合约占失败约 36.9%）。

何时必须串行

写作类任务存在强前后依赖（大纲 → 初稿 → 修订），并行子智能体会割裂上下文。Cognition 建议：把子任务当工具调用，而非对等 Agent。

来源：MAST 多智能体失败分类法（arXiv:2503.13657）；Cognition《Don't Build Multi-Agents》

多智能体质量来自独立评审与二元门控，而非单次生成



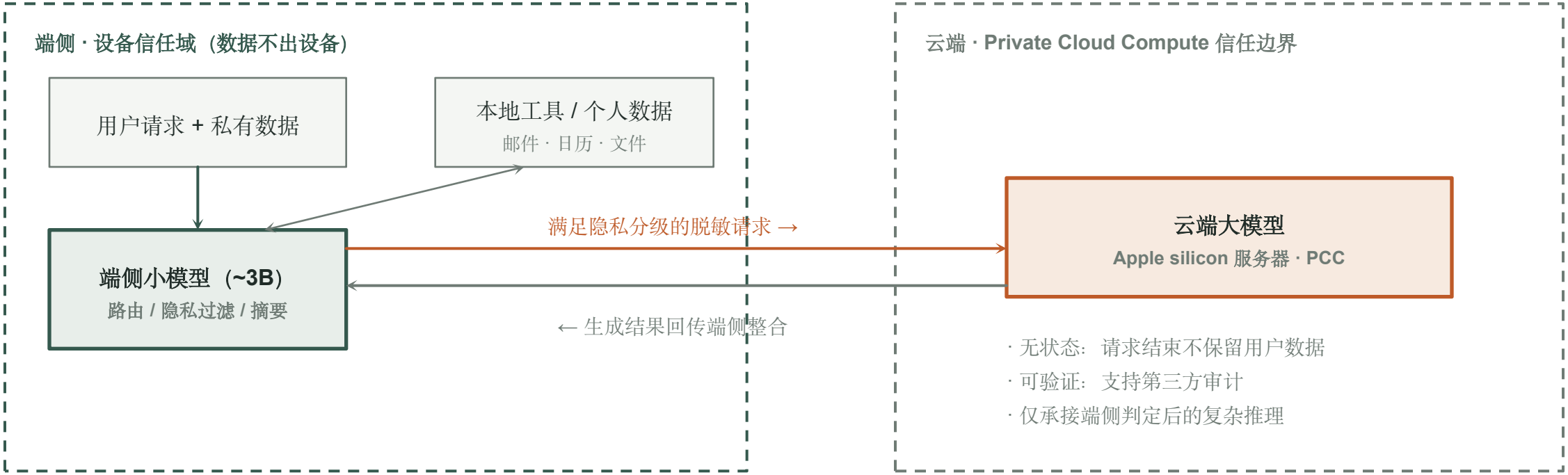
独立上下文：Reviewer 只看产物与规格，不读生成者思维链，避免自我确认

二元门控：只有 pass / fail + 结构化意见，不设「差不多通过」的中间态

验证前置：MAST 统计中任务验证类失败占约 21.3%，验证要设计进结构而非事后补丁

来源：MAST 多智能体失败分类法（arXiv:2503.13657）

推理拓扑：端侧小模型路由，云端大模型在信任边界内执行

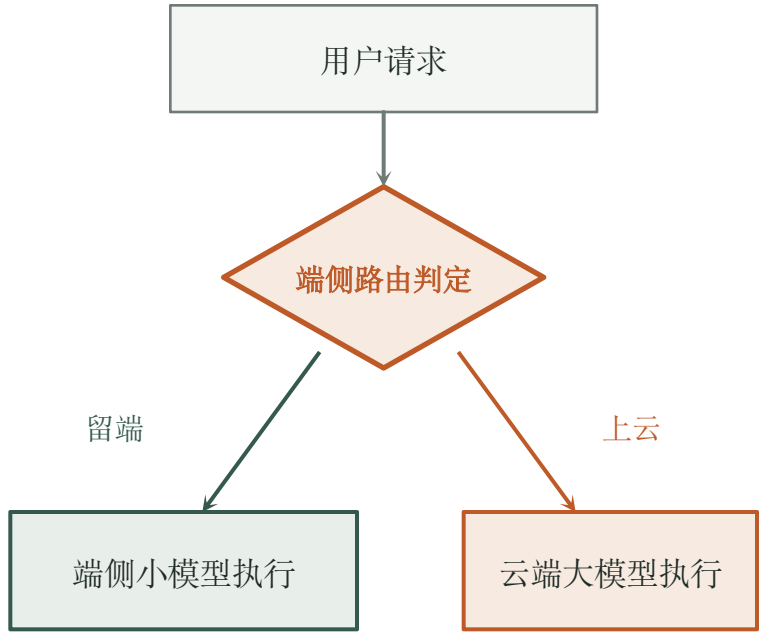


事实锚点：Apple Intelligence 端侧模型约 3B 参数，复杂请求转发运行于 Apple silicon 服务器的 PCC 大模型（无状态、可第三方审计）；2025 版端侧模型引入 KV-cache 共享与 2-bit 量化感知训练，进一步压低端侧成本。

来源：Apple Foundation Models（arXiv:2407.21075）；Apple 端侧模型技术报告 2025

任务分级决定去向：路由、隐私过滤与短文本留端侧

任务类型	去向	理由
意图识别与路由分发	留端侧	小模型即可，毫秒级响应
隐私敏感数据处理（邮件、日程）	留端侧	数据不出设备
短文本改写 / 通知摘要	留端侧	1–3B 模型质量足够
多步规划与复杂推理	上云	需要大模型长链推理
大上下文检索综合	上云	上下文与算力超端侧预算



判定依据：数据敏感度 × 所需推理深度 × 时延预算；默认留端，按需上云。

佐证：Gemini Nano 以 Nano-1（1.8B）/ Nano-2（3.25B）4-bit 量化版本在 Pixel 8 等机型离线运行，并经 AICore 向 Android 应用开放——1–3B 档模型已可承担端侧路由与过滤类任务。

来源：Gemini Nano 向 Android 开发者开放（9to5Google）；Gemini Nano 端侧部署指南

协作的代价集中在状态一致性、可观测性与 token 开销

挑战	表现（可核查事实）	工程对策
状态一致性	智能体间失合约占失败约 36.9%（MAST）；并行子智能体割裂上下文（Cognition）	单一事实源状态存储；上下文显式传递；Agent 间接口用 schema 合约固定
可观测性	MAST 归纳 3 大类 14 种失败模式；各框架任务失败率 41%–86.7%，失败难定位	全链路 trace + stage-gate 日志；每个阶段产出可校验的中间产物
token 开销	多智能体的协调与重试使 token 消耗达单 Agent 的数倍（经验口径）	端侧小模型承担路由与过滤；压缩共享上下文；按任务分级决定并行度

注：MAST 同时发现，针对失败 trace 的修补性干预仅带来约 14% 的任务完成率提升——对策必须是结构性设计，而不是提示词补丁。

来源：MAST 多智能体失败分类法（arXiv:2503.13657）；Cognition《Don't Build Multi-Agents》

CH4 · ENGINEERING PRACTICE

04

工程实践与落地案例

从架构走向工程：两个真实落地案例、一组性能数据与三次踩坑记录，以及从单点端侧优化到多 Agent 编排的三阶段落地路线。

P43 落地案例 · P44 性能与踩坑 · P45 落地路线

两个落地案例：代码审查 Agent 集群 与 百万 token 本地知识库

案例一 · 代码审查 3-Agent 集群

协作机制对应 P32 · ReAct 内环 + Handoff 外环



全程端侧运行，代码不出内网；修复建议经审计门控放行后才提交。

案例二 · 百万 token 本地知识库 RAG

1M 上下文为 Kimi K3 官方发布口径（2026-07）



多轮问答以自动摘要压缩历史上下文；疑难问题脱敏后按需上云。

优化要点：KV 分层存储控制内存峰值；多轮对话历史滚动摘要、仅保留关键结论；端侧闭环为主、按需上云，与 P28 大小脑架构共用同一套路由策略。

口径：案例与优化要点为工程实践口径 · 内部测试环境；K3 1M 上下文见 Kimi K3 发布报道（2026-07）

TTFT −35%、吞吐 +1.8×、幻觉率 ＜1%——以及三个踩过的坑

性能收益（工程实践口径）

−35%
首 Token 时延 TTFT
FlashKDA 内核 + 连续批处理

+1.8×
单卡吞吐
PD 分离 + EP 专家并行

+300%
多 Agent 任务完成率
ReAct + Handoff 协作闭环

−40~60%
Token 成本
大小脑动态卸载（P28）

15%→＜1%
幻觉率
审计门控 + 交叉验证

以上性能与成本数字均为工程实践口径 · 内部测试环境；对比基线为优化前同配置系统。

踩坑实录（现象 → 解法）

① MoE 专家坍塌

少数专家被过度路由，负载倾斜导致吞吐骤降、时延抖动。

解法：负载均衡正则，约束专家选择分布

② 端侧内存 OOM

长上下文叠加多轮对话撑爆内存，进程被系统直接回收。

解法：KV 分层存储 + 动态卸载，冷数据下沉

③ Agent 循环死锁

两个 Agent 互相等待对方输出，任务空转、白烧 token。

解法：Handoff 超时机制 + 最大步数熔断

落地三阶段：先端侧单点，再单 Agent 工具化，最后多 Agent 编排

阶段一 · 单点端侧优化

INT4 量化 + 推测解码，让目标模型在单设备达标。

门槛：decode 延迟与内存占用达交互可用线。**回退**：退回云端 API，端侧只保留路由。

1

每个阶段独立可用：回退不是失败，而是默认安全姿态。

阶段二 · 单 Agent 工具化

2

经 MCP 接入工具，跑通规划-执行-验证-集成闭环。

门槛：单任务成功率达标且 trace 完整可回放。**回退**：收窄工具集，关键节点加人工审批。

阶段三 · 多 Agent 编排

3

Orchestrator-Worker 分工，协议化接口互联。

门槛：相对单 Agent 在成功率与成本上同时占优。**回退**：退回单 Agent，子任务降级为工具调用。

验证门槛不过，不进入下一阶段。

来源：MCP（Anthropic, 2024-11）；Cognition《Don't Build Multi-Agents》（回退口径）

下一步：更低强度推理、协议标准化、工具链开源

01 K3 演进 · 更低强度推理与多模态统一

按任务难度动态调节推理深度，简单请求不浪费算力；视觉、语音与文本共用稀疏激活架构，多模态统一推理逐步下沉端侧，大小脑分工进一步细化。

02 端侧 Agent 生态 · MCP / A2A 标准化 + 联邦学习

MCP 与 A2A 分别收敛「工具接入」与「智能体互操作」，端侧 Agent 可即插即用接入生态；联邦学习让端侧小脑在数据不出设备的前提下持续进化，越用越懂本机用户。

03 开源计划 · 端侧推理工具链与协作框架社区共建

端侧推理优化工具链（量化、KV 管理、卸载策略）与 Agent 协作框架分批开源，与社区共建可复现的评测基准与工程最佳实践。

来源：MCP（Anthropic，2024-11）；A2A（Google，2025-04 发布，2025-06 捐赠 Linux Foundation）

三条判断收束：3T 模型进射程、组合拳定体验、大小脑成范式

01

KDA 混合线性注意力 + Stable LatentMoE 稀疏化，让 3T 级开源模型进入端云协同的射程

工程建议：端云协同选型优先 KDA 类线性注意力 + MoE 稀疏激活的开放权重模型，按激活参数而非总参数估算端侧成本。

02

端侧体验由组合拳决定：TurboQuant 量化、Token 稀疏化、PD 分离缺一不可

工程建议：先以 Q4 档位 + PD 分离建立基线，再逐项叠加 TurboQuant 量化与 Token 稀疏化，每步实测精度与延迟。

03

「端侧小脑 + 云端大脑」+ Handoff 协议化，是端侧 Agent 落地的工程范式

工程建议：1B-7B 端侧模型先接管感知与路由，复杂推理脱敏后经 Handoff 上云，并为每次移交设置超时与最大步数熔断。

参考来源

端侧推理与 Agents 协作都在快速演化，以下来源是复核事实与延伸阅读的起点。

01 Kimi K2 技术报告 arXiv:2507.20534

02 Kimi K3 发布分析（interconnects） interconnects.ai

03 开放权重模型清单（Qwen3 等） github.com/phlx0

04 DeepSeek 模型系列（Fireworks） fireworks.ai

05 GGUF K-quants 统一评测 arXiv:2601.14277

06 GPTQ（ICLR 2023） arXiv:2210.17323

07 AWQ（MLSys 2024） arXiv:2306.00978

08 推测解码综述 arXiv:2601.05149

09 SnapStream KV 压缩 arXiv:2511.03092

10 PowerInfer（SOSP'24） arXiv:2312.12456

11 LFM2 端侧基准方法学 arXiv:2511.23404

12 MCP 模型上下文协议 anthropic.com

13 A2A 智能体互操作协议 agenticcommerceprotocol.info

14 Cognition：Don't Build Multi-Agents cognition.ai

15 MAST 多智能体失败分类法 arXiv:2503.13657

16 Apple Foundation Models arXiv:2407.21075

17 Apple 端侧模型技术报告 2025 machinelearning.apple.com

18 Gemini Nano 向 Android 开放 9to5google.com

注：本分享中性能数据（TTFT、吞吐、幻觉率等）为演讲方工程实践口径，基于内部测试环境。